LIMO PROCESSOR
PIPELINE-REGISTER DATA MODEL

Pipeline registers store the outputs of one processor stage so that the next stage can use them during a later clock cycle. In a conventional five-stage RISC-V-style pipeline, four inter-stage registers are used: IF/ID, ID/EX, EX/MEM, and MEM/WB.

PIPELINE REGISTER FIELDS

IF/ID Register

Field	Purpose	Typical width / notes
PC	Address of the fetched instruction.	Typically 32 bits in RV32I
PC + 4	Next sequential instruction address.	Typically 32 bits
Instruction	The fetched machine instruction.	32 bits for standard RV32I instructions

ID/EX Register

Field	Purpose	Typical width / notes
PC / PC + 4	Supports branch/jump target calculations and sequential flow.	Typically 32 bits each
Read data 1	Value read from source register rs1.	32 bits
Read data 2	Value read from source register rs2.	32 bits
Immediate	Sign-extended or otherwise decoded constant/offset.	Typically 32 bits internally
rs1, rs2, rd	Source and destination register identifiers.	5 bits each for 32 registers; 4 bits for 16-register LIMO
Control signals	Select ALU operation, operand source, memory action and write-back.	Implementation-defined

EX/MEM Register

Field	Purpose	Typical width / notes
ALU result	Arithmetic/logic result or calculated memory address.	32 bits
Store data	Value to write to data memory for a store instruction.	32 bits
Destination register (rd)	Register identifier for a later write-back.	Width depends on register count
Control signals	Indicate memory read/write and write-back requirements.	Implementation-defined

Branch information (if needed)	Branch decision/target information for the chosen pipeline design.	Design-dependent

MEM/WB Register

Field	Purpose	Typical width / notes
Memory read data	Data returned by a load instruction.	32 bits
ALU result	Arithmetic/logic result or address-related result.	32 bits
Destination register (rd)	Identifies the register to receive the result.	Width depends on register count
Write-back control	Selects whether and what value is written to the register file.	Implementation-defined

How the model works

1. Instruction Fetch (IF): the processor reads the instruction using the program counter (PC).
2.IF/ID: the fetched instruction and relevant PC values are stored for decoding.
3.Instruction Decode (ID): the instruction is interpreted, source registers are read, and control signals are generated.
4.ID/EX: operands, immediate values, register identifiers and controls are stored for execution.
5.Execute (EX): the ALU performs arithmetic/logic operations, address calculations or branch comparisons.
6.EX/MEM: the ALU result, store data, destination identifier and relevant controls are stored.
7.Memory Access (MEM): load instructions read data memory; store instructions write data memory.
8.MEM/WB: memory data or the ALU result and write-back information are stored.
9.Write Back (WB): the selected result is written to the destination register when required.